

overload protection

what is it and why is it needed

context **2025** meeting

What is it

- In principle a user can any moment change the way T_EX works.
- This can be intentional or by accident.
- In doing so side effects can get unnoticed.
- Candidates for change are T_EX macros and registers but also Lua and MetaPost code.
- So how can we prevent harm being done.

How does it work: macros

Users can define macros:

```
\def\MyMacro{do something}
```

But what if that one is also a system one?

```
\permanent\def\MyMacro{do something}
```

This tags a macro as one that can't be changed.

```
\overloadmode=4 % warning
```

```
\overloadmode=5 % error
```

How does it work: variables

Users can define variables:

```
\integerdef\MyInteger 123
```

These definitions can also be protected:

```
\permanent\integerdef\MyInteger 0
```

But values can still be changed unless we say:

```
\immutable\MyInteger 123
```

From then on they can't be changed. The same applies to for instance `\countdef`.

The `\newinteger` and `\newcount` definition macros already make the target permanent.

Impact on engine

- We need to relate states to all entries in the so called table of equivalents.
- Quite some properties are set by prefixes; we can group them.
- Obsolete: `\long`, `\outer`.
- Backend: `\deferred`, `\immediate`.
- Scope: `\global`.
- Expansion: `\protected`, `\semiprotected`.
- Kind of special: `\noaligned`, `\untraced`, `\constrained`, `\retained`, `\tolerant`, `\constant`.
- Protection: `\aliased`, `(\always)`, `(\enforced)`, `\frozen`, `\immutable`, `\inherited`, `\instance`, `\mutable`, `\overloaded`, `\permanent`.

Candidates for protection

arithmic	<code>\advance</code>	def	<code>\def</code>
internal integer	<code>\hangafter</code>	define family	<code>\scriptfont</code>
register integer	<code>\countdef</code>	define font	<code>\font</code>
internal attribute	(none)	hyphenation	<code>\hjcode</code>
register attribute	<code>\attributedef</code>	let	<code>\let</code>
internal posit	(none)	prefix	<code>\global</code>
register posit	<code>\floatdef</code>	register	<code>\count</code>
internal dimension	<code>\hangindent</code>	auxiliary	<code>\prevgraf</code>
register dimension	<code>\dimendef</code>	set box	<code>\setbox</code>
font property	<code>\efcode</code>	box property	<code>\wd</code>
internal glue	<code>\baselineskip</code>	set font	<code>\nullfont</code>
register glue	<code>\skipdef</code>	interaction	<code>\batchmode</code>
internal muglue	<code>\medmuskip</code>	math parameter	<code>\setmathatomrule</code>
register muglue	<code>\muskipdef</code>	page property	<code>\pagegoal</code>
internal toks	<code>\everypar</code>	specification	<code>\parshape</code>
register toks	<code>\toksdef</code>	shorthand def	<code>\chardef</code>
define char code	<code>\catcode</code>	association	<code>\associateunit</code>

lua value	(in lua)	gluespec	\gluespecdef
integer	\integerdef	mugluespec	\mugluespecdef
posit	\positdef	index	\parameterdef
dimension	\dimensiondef	combine toks	\toksapp

We need to deal with protection at the T_EX end as well as at the Lua end.

In Lua we can to some extent discourage overloading of functionality and local binding is also an option.

In MetaPost we also have some mechanism but for now we don't error there (conceptually different grouping model).

Virus protection

Can T_EX ever be really secured?

- Because T_EX reads and writes files, much depends on trust.
- It can never be made complete safe but to some extend we can at least try and give the user a good feeling.
- Can we be sure that T_EXies are the ‘good guys’?
- In ConT_EXt we can sandbox, that is, all tricky mechanisms have intercepts and can be controlled from the configuration. No one ever does that.
- The community can impose some restrictions and/or guard integrity of macro packages. As it contradicts open source there is no such policy.
- Can we intercept injection of potential harmful code? Because one can adapt the source early on there is little we can do. So it depends.

- To some extent overload protection can help but bad (also T_EX) actors can always get around it. Unless we take more drastic measures.
- In that respect it is like virus scanners (operating system, browsers, apps on phones, etc): the best you can do is observe and adapt.